

Adaptive Game Difficulty Agent

Dynamische Schwierigkeitsanpassung durch Spielsimulation

Ufuk Ofoglu
Modul: Angewandte Modellierung und Systemsimulation

25. Juli 2026

GitHub: <https://github.com/Ofoglufuu/Adaptive-Game-Difficulty-Agent>

1 Motivation und Forschungsfrage

In Videospielen kann ein statischer Schwierigkeitsgrad dazu führen, dass Anfänger überfordert und erfahrene Spieler unterfordert sind. Die dynamische Schwierigkeitsanpassung (Dynamic Difficulty Adjustment, DDA) löst dieses Problem, indem sie den Schwierigkeitsgrad während der Laufzeit auf Basis der beobachteten Spielerleistung anpasst. In diesem Projekt wird eine vereinfachte probabilistische Spielsimulation anstelle einer echten Game Engine verwendet, um dieses Konzept zu evaluieren.

Die zentrale Forschungsfrage lautet:

„Kann ein adaptiver Schwierigkeitsagent eine ausgewogenere Spielerfahrung erzeugen als ein statisches Schwierigkeitssystem?“

Um die Ausgewogenheit zu messen, wird eine Zielgewinnrate von 60% als vereinfachter Indikator verwendet. Dabei ist zu beachten, dass die reine Gewinnrate nur ein Proxy für die Spielbalance ist und Aspekte wie Spielspaß, Fairness oder die umfassende Spielerfahrung nicht vollständig abbilden kann.

2 Architektur und Tech Stack

Die Architektur des Systems ist auf Determinismus und Nachvollziehbarkeit ausgelegt. Die Kernkomponenten kommunizieren über direkte Python-Funktionsaufrufe.

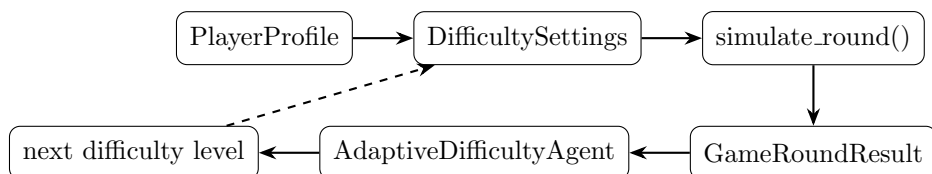


Abbildung 1: Ablauf der Kernsimulation

Darüber hinaus existiert ein optionaler Pfad für den interaktiven Modus:

reaction test → median reaction time → reaction score → personalized PlayerProfile → adaptive live demo

Die Umsetzung der verschiedenen architektonischen Schichten gestaltet sich wie folgt:

Bereich	Umsetzung
Backend / Simulation	Selbst entwickelte probabilistische Simulation in Python
Intelligence	Transparenter regelbasierter AdaptiveDifficultyAgent
Orchestrierung	Direkte Python-Funktionsaufrufe über <code>main.py</code> und <code>experiment.py</code>
Auswertung	CSV-Export, Matplotlib-Diagramme und Pytest
Interaktion	Terminal-Live-Demo und optionaler Reaktionstest

Tabelle 1: Konzepte und Technologien

Architekturentscheidungen und Protokolle: Das System ist bewusst als lokales Single-Agent-System konzipiert. Es verwendet weder das Model Context Protocol (MCP) noch Agent-to-Agent-Kommunikation (A2A). Die Module kommunizieren ausschließlich über direkte Python-Funktionsaufrufe. Es sind weder Large Language Models (LLM) noch ein Agent Development Kit (ADK) erforderlich. Diese Architekturentscheidung wurde getroffen, da das Projekt höchste Priorität auf Transparenz, deterministisches Verhalten, Reproduzierbarkeit und einen überschaubaren technischen Umfang legt.

3 Simulationsmodell und Agentenlogik

Für die Experimente werden drei vordefinierte, synthetische Spielerprofile verwendet, die unterschiedliche Leistungsniveaus repräsentieren:

Profil	skill_level	accuracy	reaction_speed	survival_factor
Anfänger	0.35	0.30	0.25	0.40
durchschnittlicher Spieler	0.60	0.60	0.55	0.65
erfahrener Spieler	0.85	0.85	0.80	0.90

Tabelle 2: Synthetische Spielerprofile

Der Schwierigkeitsgrad (`level`) reicht von 1 bis 10 und bestimmt drei Spielparameter:

- `enemy_count` = $3 + (\text{level} - 1)$
- `enemy_damage` = $5 + (\text{level} - 1)$
- `health_pack_rate` = $0.40 - 0.03 \times (\text{level} - 1)$

Die Heilungswahrscheinlichkeit (`health_pack_rate`) wird in der Implementierung gerundet. Ein höherer Schwierigkeitsgrad bedeutet folglich mehr Gegner, höheren Gegnerschaden und eine geringere Heilungswahrscheinlichkeit.

Der `AdaptiveDifficultyAgent` hat folgende Standardeinstellungen: Zielgewinnrate: 60 %, Beobachtungsfenster: letzte 5 Runden, Min./Max. Schwierigkeit: 1 bis 10, maximale Änderung: eine Stufe pro Entscheidung. Es handelt sich um einen reinen regelbasierten Agenten (kein Machine Learning). Die Entscheidungsregeln werden in dieser Reihenfolge ausgewertet:

1. Drei aufeinanderfolgende Niederlagen → reduzieren
2. Drei aufeinanderfolgende Siege und Gewinnrate über Ziel → erhöhen
3. Gewinnrate $\leq 40\%$ → reduzieren
4. Gewinnrate $\geq 80\%$ → erhöhen
5. Sonst → beibehalten

4 Experimentdesign und Ergebnisse

Das Standard-Vergleichsexperiment testet das statische System (konstanter Schwierigkeitsgrad) gegen das adaptive System (Auswertung nach jeder Runde). Es nutzt die drei Spielerprofile, drei feste Seeds (42, 123, 999) und 300 Runden pro Experiment. Dies resultiert in insgesamt 18 Experimentdurchläufen und 5400 simulierten Runden. Die Startschwierigkeit ist stets 5, die Zielgewinnrate beträgt 60 %. Die primäre Metrik ist die absolute Abweichung von diesem Ziel.

Die folgenden mittleren Ergebnisse wurden ermittelt:

Profil	Statisch: Gewinnrate	Adaptiv: Gewinnrate	Statisch: Zielabweichung	Adaptiv: Zielabweichung
Anfänger	35.1 %	50.2 %	24.9 Prozentpunkte	9.8 Prozentpunkte
durchschnittlicher Spieler	66.7 %	52.3 %	6.7 Prozentpunkte	7.7 Prozentpunkte
erfahrener Spieler	88.0 %	56.0 %	28.0 Prozentpunkte	4.0 Prozentpunkte

Tabelle 3: Mittlere Ergebnisse der Vergleichsexperimente

Interpretation: Für den Anfänger reduziert das adaptive System die anfängliche Überforderung, wodurch das Ergebnis näher am Ziel liegt. Beim erfahrenen Spieler erhöht das System den anfänglich zu niedrigen Schwierigkeitsgrad, was ebenfalls zu einer deutlich geringeren Zielabweichung führt. Für den durchschnittlichen Spieler ist die Startschwierigkeit 5 jedoch bereits sehr gut kalibriert, sodass das statische System leicht besser abschneidet. Das adaptive System ist folglich besonders dann nützlich, wenn die Startschwierigkeit nicht optimal zum Spieler passt; es ist jedoch nicht unter allen Umständen überlegen.

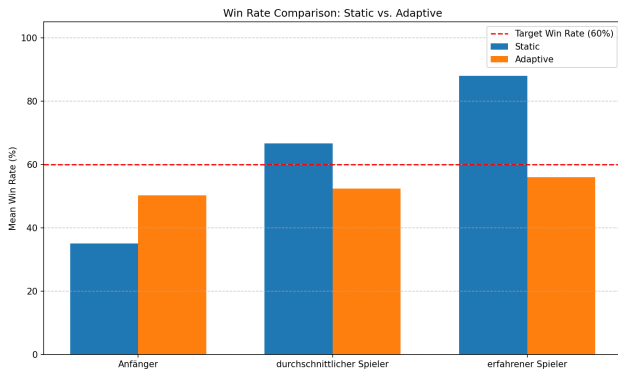


Abbildung 2: Gewinnraten im Vergleich

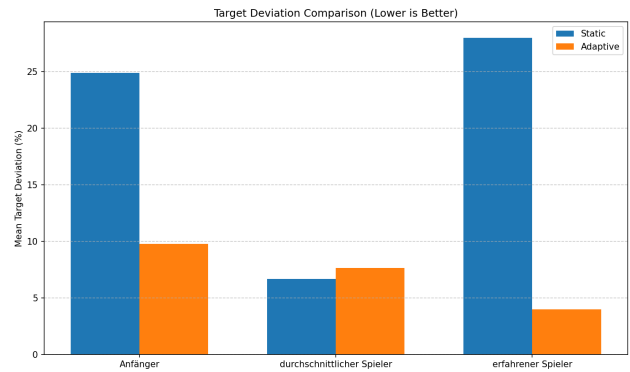


Abbildung 3: Abweichung von der Zielgewinnrate

5 Interaktive Erweiterung, Qualitätssicherung und Fazit

Reaktionstest-Modus: Als optionales interaktives Feature (nicht Teil des wissenschaftlichen Kernvergleichs) kann der Nutzer im Terminal Reaktionsmessungen durchführen. Aus dem Median dieser Messungen wird ein Score zwischen 0.0 und 1.0 gebildet. Dieser personalisiert ausschließlich den Wert `reaction_speed`. Die übrigen Parameter (`skill_level` = 0.60, `accuracy` = 0.60, `survival_factor` = 0.65) bleiben vordefiniert. Messungen unter 80 ms werden als implausibel frühe oder gepufferte Eingaben abgelehnt und der jeweilige Versuch wird wiederholt. Die projektspezifischen Labels `expert`, `average` und `beginner` werden ausschließlich für diese Demo verwendet und stellen keine wissenschaftliche Bewertung der allgemeinen Spielfähigkeit dar.

Qualitätssicherung: Die Integrität des Projekts wird durch 113 bestandene automatisierte Pytest-Tests sichergestellt. Die Verwendung fester Seeds macht die Simulation reproduzierbar, was bedeutet, dass der CSV-Output mit denselben Parametern identisch bleibt. Die Reaktionsmessungen hingegen bleiben zwangsläufig von menschlichen Eingaben und Latenzen des Geräts abhängig.

Limitationen: Das Projekt weist mehrere Limitationen auf: Es verwendet eine vereinfachte probabilistische Simulation anstelle einer echten Game Engine und arbeitet mit synthetischen Profilen anstelle echter Spieltelemetrie. Die Gewinnrate ist lediglich ein Proxy für die Spielbalance. Der Agent ist strikt regelbasiert und lernt nicht selbstständig. Zudem ist der Reaktionstest anfällig für Latenzen (Tastatur, Display, Terminal) sowie die Aufmerksamkeit des Nutzers.

Fazit: Innerhalb des vereinfachten Modells kann der adaptive Agent ein ausgewogeneres Ergebnis erzeugen, insbesondere dann, wenn die statische Startschwierigkeit für den jeweiligen Spieler ungeeignet ist. Allerdings kann ein gut gewählter statischer Schwierigkeitsgrad bereits sehr gut abschneiden, wie das Profil des durchschnittlichen Spielers zeigt. Das Projekt demonstriert somit das Potenzial einer transparenten dynamischen Schwierigkeitsanpassung, ohne deren Effektivität in kommerziellen Spielen abschließend zu belegen.